

Praktische Übung: CQRS und Event Sourcing mit Node.js

Modul 321

5. Mai 2026

Ziel der Übung

In dieser Übung entwickelt ihr ein einfaches Bankkonto-System und wendet dabei zwei wichtige Architekturkonzepte an:

- **CQRS (Command Query Responsibility Segregation)**
- **Event Sourcing**

Wichtig: Der Kontostand darf niemals direkt gespeichert werden! Stattdessen werden alle Änderungen als **Events** gespeichert.

Funktionalität

Euer System soll:

- Konto eröffnen
- Geld einzahlen
- Geld abheben
- Kontostand anzeigen
- Alle Events anzeigen
- Zustand durch Events rekonstruieren

Projektstruktur

```
cqrs-event-sourcing-bank/  
|  
|-- data/  
|   '-- events.jsonl  
|  
'-- src/  
    |-- eventStore.js
```

```
-- account.js
-- commands.js
-- queries.js
'-- app.js
```

JSONL Format

Jede Zeile ist ein Event:

```
{"type": "AccountOpened", "accountId": "A-100", "owner": "Mia"}
{"type": "MoneyDeposited", "accountId": "A-100", "amount": 100}
```

Aufgabe 1: Event Store

Implementiert:

- `appendEvent(event)`
- `readEvents()`

Aufgabe 2: Zustand berechnen

Startzustand:

```
{
  accountId: null,
  owner: null,
  balance: 0,
  isOpen: false
}
```

Implementiert:

- `applyEvent(account, event)`
- `rebuildAccountFromEvents(events, accountId)`

Aufgabe 3: Commands

Implementiert:

- `openAccount(accountId, owner)`
- `depositMoney(accountId, amount)`
- `withdrawMoney(accountId, amount)`
- `closeAccount(accountId)`

Wichtig: Commands verändern niemals direkt den Zustand!
Sie erzeugen nur Events.

Aufgabe 4: Queries

Implementiert:

- `getAccount(accountId)`
- `getAllEvents()`

Wichtig: Queries verändern keine Daten.

Aufgabe 5: Testprogramm

Beispiel:

```
openAccount("A-100", "Mia");  
depositMoney("A-100", 200);  
withdrawMoney("A-100", 50);  
  
console.log(getAccount("A-100"));
```

Beobachtungsaufgaben

Beantwortet:

1. Wird der Kontostand direkt gespeichert?
2. Welche Events entstehen?
3. Kann man den Zustand aus Events rekonstruieren?
4. Was passiert bei mehrfacher Ausführung?

CQRS verstehen

Command-Seite:

- `openAccount`
- `depositMoney`
- `withdrawMoney`
- `closeAccount`

Query-Seite:

- `getAccount`
- `getAllEvents`

Erweiterung: Read Model

Erstellt eine Datei:

```
accounts_read_model.json
```

Diese darf den Kontostand speichern, wird aber aus Events berechnet.

Reflexionsfragen

1. Warum wird der Kontostand nicht direkt gespeichert?
2. Was ist der Vorteil von Event Sourcing?
3. Was ist ein Nachteil?
4. Warum trennt man Commands und Queries?
5. Was passiert, wenn das Read Model gelöscht wird?
6. Kann man es wiederherstellen?

Zusatzaufgaben

Mehrere Konten

Erstellt mehrere Konten und zeigt sie an.

Überweisung

Implementiert:

```
transferMoney(from, to, amount)
```

REST API

Optional mit Express:

```
POST /accounts
POST /accounts/:id/deposit
GET /accounts/:id
```

Bewertung

- Events werden korrekt gespeichert
- Trennung von Commands und Queries
- Zustand wird aus Events berechnet
- Fehler werden behandelt
- Struktur ist verständlich

Diskussion

- Wann ist Event Sourcing sinnvoll?
- Wann ist es zu kompliziert?
- Welche Systeme könnten davon profitieren?